POO (Java) — TP1

A.U. 2026–2027

Exercice 1 : Gestion d'une bibliothèque

Créer une classe *Livre* avec titre, auteur, ISBN et disponibilité, ainsi qu'un constructeur, des getters/setters, les méthodes `emprunter()`, `retourner()` et `toString()`. Créer ensuite *Bibliotheque*, avec un tableau fixe de livres, les méthodes d'ajout, d'emprunt par ISBN et d'affichage des livres disponibles. Tester dans une classe *Main*.

Livre.java

```java
public class Livre {
    private String titre;
    private String auteur;
    private String isbn;
    private boolean disponible=true;

    public Livre(String titre, String auteur, String isbn) {
        this.titre=titre;
        this.auteur=auteur;
        this.isbn=isbn;
    }

    public String getTitre() { return titre; }
    public void setTitre(String titre) { this.titre=titre; }
    public String getAuteur() { return auteur; }
    public void setAuteur(String auteur) { this.auteur=auteur; }
    public String getIsbn() { return isbn; }
    public void setIsbn(String isbn) { this.isbn=isbn; }
    public boolean isDisponible() { return disponible; }
    public void setDisponible(boolean disponible) { this.disponible=disponible; }

    public boolean emprunter() {
        if(!disponible) {
            return false;
        }
        disponible=false;
        return true;
    }

    public void retourner() {
        disponible=true;
    }

    @Override
    public String toString() {
        return titre + " - " + auteur + " (ISBN : " + isbn + ") - "
                + (disponible ? "disponible" : "emprunte");
    }
}
```

Bibliotheque.java

```java
public class Bibliotheque {
    private final Livre[] livres;
    private int nombreLivres;

    public Bibliotheque(int capacite) {
        livres=new Livre[capacite];
    }

    public boolean ajouterLivre(Livre livre) {
        if(nombreLivres==livres.length) {
            System.out.println("La bibliotheque est pleine.");
            return false;
        }
        livres[nombreLivres++]=livre;
        return true;
    }

    public boolean emprunterLivre(String isbn) {
        for(int i=0;i<nombreLivres;i++) {
            if(livres[i].getIsbn().equals(isbn)) {
                return livres[i].emprunter();
            }
        }
        return false;
    }

    public void afficherLivresDisponibles() {
        System.out.println("Livres disponibles :");
        for(int i=0;i<nombreLivres;i++) {
            if(livres[i].isDisponible()) {
                System.out.println("- " + livres[i]);
            }
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Bibliotheque bibliotheque=new Bibliotheque(5);
        bibliotheque.ajouterLivre(new Livre("L'Etranger", "Albert Camus", "978-2070360024"));
        bibliotheque.ajouterLivre(new Livre("1984", "George Orwell", "978-2070368228"));
        bibliotheque.ajouterLivre(new Livre("Le Petit Prince", "Antoine de Saint-Exupery", "978-
2070612758"));

        bibliotheque.emprunterLivre("978-2070368228");
        bibliotheque.afficherLivresDisponibles();
    }
}
```

Exercice 2 : Compte bancaire simple

Créer *CompteBancaire* avec le numéro, le titulaire et le solde. Les méthodes de dépôt et retrait doivent vérifier les montants. Créer *Banque* avec un tableau fixe de comptes, l'ouverture, la recherche, le virement et l'affichage des comptes. Tester dans une classe *Main*.

CompteBancaire.java

```
public class CompteBancaire {
    private final String numeroCompte;
    private final String titulaire;
    private double solde;

    public CompteBancaire(String numeroCompte, String titulaire) {
        this.numeroCompte=numeroCompte;
        this.titulaire=titulaire;
        solde=0;
    }

    public boolean deposer(double montant) {
        if(montant<=0) {
            return false;
        }
        solde+=montant;
        return true;
    }

    public boolean retirer(double montant) {
        if(montant<=0 || montant>solde) {
            return false;
        }
        solde-=montant;
        return true;
    }

    public void afficherSolde() {
        System.out.printf("Compte %s : %.2f DT%n", numeroCompte, solde);
    }

    public String getNumeroCompte() { return numeroCompte; }
    public String getTitulaire() { return titulaire; }
    public double getSolde() { return solde; }
}
```

Banque.java

```java
public class Banque {
    private final CompteBancaire[] comptes;
    private int nombreComptes;

    public Banque(int capacite) {
        comptes=new CompteBancaire[capacite];
    }

    public boolean ouvrirCompte(CompteBancaire compte) {
        if(nombreComptes==comptes.length || rechercherCompte(compte.getNumeroCompte())!=null) {
            return false;
        }
        comptes[nombreComptes++]=compte;
        return true;
    }

    public CompteBancaire rechercherCompte(String numeroCompte) {
        for(int i=0;i<nombreComptes;i++) {
            if(comptes[i].getNumeroCompte().equals(numeroCompte)) {
                return comptes[i];
            }
        }
        return null;
    }

    public boolean effectuerVirement(String numSource, String numDest, double montant) {
        CompteBancaire source=rechercherCompte(numSource);
        CompteBancaire destination=rechercherCompte(numDest);
        if(source==null || destination==null || source==destination || montant<=0) {
            return false;
        }
        if(!source.retirer(montant)) {
            return false;
        }
        destination.deposer(montant);
        return true;
    }

    public void afficherTousLesComptes() {
        for(int i=0;i<nombreComptes;i++) {
            System.out.printf("%s (%s) : %.2f DT%n", comptes[i].getNumeroCompte(),
                    comptes[i].getTitulaire(), comptes[i].getSolde());
        }
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Banque banque=new Banque(10);
        CompteBancaire compte1=new CompteBancaire("C001", "Amira Ben Ali");
        CompteBancaire compte2=new CompteBancaire("C002", "Youssef Trabelsi");
        banque.ouvrirCompte(compte1);
        banque.ouvrirCompte(compte2);

        compte1.deposer(500);
        banque.effectuerVirement("C001", "C002", 125.5);
        banque.afficherTousLesComptes();
    }
}
```

Exercice 3 : Application console

Dans un fichier source compact, utiliser `void main()` et la classe `IO` de `java.lang` pour lire N, afficher un message personnalisé puis la somme des entiers de 1 à N.

SommeEntiers.java

```java
// Necessite Java 25 ou une version recente avec les fichiers source compacts.
void main() {
    String saisie=IO.readln("Donnez N : ");
    int n;

    try {
        n=Integer.parseInt(saisie);
    } catch (NumberFormatException e) {
        IO.println("Veuillez saisir un entier valide.");
        return;
    }

    if(n<1) {
        IO.println("N doit etre superieur ou egal a 1.");
        return;
    }

    long somme=0;
    for(int i=1;i<=n;i++) {
        somme+=i;
    }
    IO.println("Bonjour ! La somme de 1 a " + n + " est " + somme + ".");
}
```

Exercice 4 : Flexible Constructor Bodies

Créer *Produit* avec un nom et un prix. Vérifier que le prix est strictement positif dans le prologue du constructeur, avant l'appel à `super()`. Ajouter un constructeur qui délègue au premier avec `this(...)`.

Produit.java

```java
// Necessite Java 25 : Flexible Constructor Bodies.
public class Produit {
    private final double prix;
    private final String nom;

    public Produit(String nom, double prix) {
        if(prix<=0) {
            throw new IllegalArgumentException("Le prix doit etre strictement positif.");
        }
        super();
        this.nom=nom;
        this.prix=prix;
    }

    public Produit(double prix) {
        this("Produit sans nom", prix);
    }

    @Override
    public String toString() {
        return nom + " : " + prix + " DT";
    }
}
```

Main.java

```java
public class Main {
    public static void main(String[] args) {
        Produit p1=new Produit("Clavier", 89.9);
        Produit p2=new Produit(12.5);
        System.out.println(p1);
        System.out.println(p2);
    }
}
```

Exercice 5 : Module Import Declarations

Écrire deux fichiers compacts utilisant `import module java.base;`. Le premier calcule l'aire et le périmètre d'un cercle ; le second calcule la moyenne d'un tableau d'entiers.

Geometrie.java

```java
import module java.base;

void main() {
    double rayon=4.5;
    double perimetre=2*Math.PI*rayon;
    double aire=Math.PI*rayon*rayon;

    IO.println(String.format("Rayon : %.2f", rayon));
    IO.println(String.format("Perimetre : %.2f", perimetre));
    IO.println(String.format("Aire : %.2f", aire));
}
```

Statistiques.java

```java
import module java.base;

void main() {
    int[] notes={12, 15, 8, 10, 17};
    int total=0;

    for(int note : notes) {
        total+=note;
    }

    double moyenne=(double)total/notes.length;
    IO.println(String.format("Moyenne : %.2f", moyenne));
}
```

Exercice 6 : Calcul de statistiques sur les températures

Lire de 3 à 30 températures entre −50 et 60 °C, en redemandant les valeurs invalides. Calculer la moyenne, le minimum, le maximum, l'amplitude, l'écart-type, les jours de gel et la médiane avec un tri à bulles.

Temperatures.java

```java
import java.util.Locale;
import java.util.Scanner;

public class Temperatures {
    public static void main(String[] args) {
        Scanner scanner=new Scanner(System.in);
        scanner.useLocale(Locale.US);
        int n=lireNombreTemperatures(scanner);
        double[] temperatures=new double[n];
        double somme=0;
        double minimum=60;
        double maximum=-50;
        int joursGel=0;

        for(int i=0;i<n;i++) {
            temperatures[i]=lireTemperature(scanner, i+1);
            somme+=temperatures[i];
            if(temperatures[i]<minimum) minimum=temperatures[i];
            if(temperatures[i]>maximum) maximum=temperatures[i];
            if(temperatures[i]<0) joursGel++;
        }

        double moyenne=somme/n;
        double sommeEcarts=0;
        for(double temperature : temperatures) {
            double ecart=temperature-moyenne;
            sommeEcarts+=ecart*ecart;
        }

        double[] copie=temperatures.clone();
        trierBulles(copie);
        double mediane=n%2==0 ? (copie[n/2-1]+copie[n/2])/2 : copie[n/2];

        System.out.println("--- Resultats ---");
        System.out.printf("Moyenne     : %.2f °C%n", moyenne);
        System.out.printf("Maximum     : %.2f °C%n", maximum);
        System.out.printf("Minimum     : %.2f °C%n", minimum);
        System.out.printf("Amplitude   : %.2f °C%n", maximum - minimum);
        System.out.printf("Ecart-type  : %.2f%n", Math.sqrt(sommeEcarts / n));
        System.out.printf("Mediane     : %.2f °C%n", mediane);
        System.out.println("Jours de gel : " + joursGel);
        scanner.close();
    }

    private static int lireNombreTemperatures(Scanner scanner) {
        while(true) {
            System.out.print("Combien de temperatures ? ");
            if(scanner.hasNextInt()) {
                int n=scanner.nextInt();
                if(n>=3 && n<=30) return n;
            } else {
                scanner.next();
            }
            System.out.println("Saisissez un entier entre 3 et 30.");
        }
    }

    private static double lireTemperature(Scanner scanner, int numero) {
        while(true) {
            System.out.print("Temperature " + numero + " : ");
            if(scanner.hasNextDouble()) {
                double valeur=scanner.nextDouble();
                if(valeur>=-50 && valeur<=60) return valeur;
            } else {
                scanner.next();
            }
            System.out.println("La temperature doit etre comprise entre -50 et 60.");
        }
    }

    private static void trierBulles(double[] valeurs) {
        for(int fin=valeurs.length-1;fin>0;fin--) {
            for(int i=0;i<fin;i++) {
                if(valeurs[i]>valeurs[i+1]) {
                    double temporaire=valeurs[i];
                    valeurs[i]=valeurs[i+1];
                    valeurs[i+1]=temporaire;
                }
            }
        }
```

```
    }
}
```

Exercice 7 : Étudiants admis

Créer le record *Student*. Afficher les étudiants admis sans Stream, puis avec l'API Stream, `Gatherers.windowFixed(...)` et le pattern matching pour les records.

Student.java

```java
public record Student(String name, double average) {
}
```

EtudiantsAdmis.java

```java
import java.util.List;
import java.util.stream.Gatherers;

public class EtudiantsAdmis {
    public static void main(String[] args) {
        List<Student> etudiants=List.of(
                new Student("Amina", 14.5),
                new Student("Karim", 9.75),
                new Student("Sana", 10.0),
                new Student("Mehdi", 7.5),
                new Student("Rim", 16.25));

        System.out.println("Sans Stream :");
        afficherAdmisSansStream(etudiants);

        System.out.println("\nAvec Stream et Gatherers :");
        etudiants.stream()
                .filter(etudiant -> etudiant.average() >= 10.0)
                .gather(Gatherers.windowFixed(2))
                .forEach(groupe -> {
                    for(Student etudiant : groupe) {
                        afficherAdmission(etudiant);
                    }
                });
    }

    private static void afficherAdmisSansStream(List<Student> etudiants) {
        for(Student etudiant : etudiants) {
            if(etudiant.average()>=10.0) {
                afficherAdmission(etudiant);
            }
        }
    }

    private static void afficherAdmission(Student etudiant) {
        Object valeur=etudiant;
        if(valeur instanceof Student(String nom, double moyenne)) {
            System.out.printf("L'etudiant %s est admis avec une moyenne de %.2f/20.%n", nom, moyenne);
        }
    }
}
```